MIT | Academy of Engineering
(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

Report

Feature Engineering

Assignment No. 1: Implement techniques of Feature Engineering to transforms raw data into informative features

Name: Arjun Mankuskar

PRN:202401110038

Branch: CSE (AIML)

Batch: A2

Date: November 2, 2025

1. Introduction & Objective

For this project, my goal was to find patterns in data using machine learning. But a model can only find patterns in the data it's given, which is why preparing that data (feature engineering) is the most critical part.

My objective was to build and analyze two totally different feature engineering pipelines. I picked two public datasets from Kaggle: one for image classification and one for text. This report goes over the datasets, the techniques I used, why I chose them, and what happened as a result.

2. Part 1: Image Dataset (Computer Vision)

- **Dataset Chosen: The "Dog vs. Cat" dataset from Kaggle. This is a classic binary classification problem. The version I used had 1,000 images, balanced with 500 cats and 500 dogs.**

Feature Engineering Techniques Applied

Task	Technique Used
Loading & Label Encoding	tf.keras.utils.image_dataset_from_directory
Feature Scaling	tf.keras.applications.mobilenet_v2.preprocess_input
Feature Extraction	Transfer Learning (using a pre-trained MobileNetV2 base)
Feature Selection	tf.keras.layers.GlobalAveragePooling2D

Reasoning Behind Choices

For image data, the "pipeline" is different from a simple table. I used a modern deep learning workflow:

1. **Loading & Encoding: I went with TensorFlow's image_dataset_from_directory utility because it saved a ton of time. It loaded all the images and automatically handled the labels. It saw the cat and dog folders and knew to encode them as 0 and 1.**
2. **Scaling: Neural networks work much better with small, centered numbers. So, I had to scale the pixels. I used the specific preprocess_input function for the MobileNetV2 model, which is a specialized Normalization that scales all pixel values from the standard [0, 255] range down to [-1, 1].**
3. **Extraction: Training a new image model from scratch takes forever, so I used Transfer Learning. This is the standard solution. I loaded a MobileNetV2 model that was pre-trained on millions of images. By "freezing" its layers, I could use it as a powerful feature extractor that already knows how to find edges, textures, and shapes.**

4. **Selection: The pre-trained model outputs a large block of features. I used a GlobalAveragePooling2D layer to "select" the most dominant features by averaging them. This flattens the large block into a single, manageable vector that the final classifier can use.**

Observations on Transformed Features

This pipeline worked really well. Each image, which is just a big grid of pixels, was transformed into a 1,280-feature vector that actually represents what's in the picture. These features were so high-quality that a simple classifier (with only about 1,200 trainable parameters) was able to get 100% validation accuracy. This result perfectly shows that transfer learning is a highly effective feature engineering technique.

3. Part 2: Text Dataset (NLP)

- **Dataset Chosen: The "IMDB 50K Movie Reviews" dataset from Kaggle. This is a large, balanced dataset for binary sentiment classification (25,000 positive and 25,000 negative reviews).**

Feature Engineering Techniques Applied

Task	Technique Used
Load & Check Data	pandas.read_csv and df.isnull().sum()
Data Cleaning	Regular Expressions (re) and nltk.stopwords
Label Encoding	sklearn.preprocessing.LabelEncoder
Feature Extraction	sklearn.feature_extraction.text.TfidfVectorizer
Feature Scaling	TfidfVectorizer (default L2 Normalization)
Feature Selection	sklearn.feature_selection.SelectKBest (with chi2 test)

Reasoning Behind Choices

This was a more classic, multi-step NLP pipeline:

1. **Load & Check: First, I loaded the data into a pandas DataFrame. My first check with .isnull().sum() showed zero missing values. This was great because it meant I could skip any imputation steps.**
2. **Cleaning: The raw text was a mess. It was full of HTML tags (
), punctuation, and common "stop words" (like 'the', 'is', 'a') that just add noise. I wrote a function to remove all of this junk and convert everything to lowercase.**
3. **Encoding: The model can't read "positive" or "negative." I used LabelEncoder to convert these text labels into 1s and 0s.**

4. **Extraction (TF-IDF): To turn the cleaned words into numbers, I used TfidfVectorizer. This is much smarter than a simple word count. It scores words based on how *important* they are to a specific review (not just how often they appear). I limited this to the top 5,000 features.**
5. **Scaling: This step was conveniently handled by the TfidfVectorizer. It automatically applies L2 Normalization to its output, which ensures all my feature vectors are on a consistent scale.**
6. **Selection: 5,000 features is still a lot, so I used SelectKBest with a chi2 (Chi-Squared) test. This helped me find the top 2,000 features that had the strongest statistical relationship with the positive/negative label.**

Observations on Transformed Features

The pipeline was a complete success. It transformed 50,000 messy, unstructured text reviews into a clean, normalized, and predictive (40000, 2000) matrix (for the training set). I tested this by training a simple Naive Bayes model on the final data, which achieved ~88% accuracy on the test set. This proves the raw text was successfully engineered into a set of features the model could learn from.

4. Ethical Concerns and Mitigation

The assignment also asked about the ethics of using features like 'Gender' or 'Marital Status' to predict employee attrition.

- **The Problem: Using features like these is a really bad idea. It's a textbook example of systemic bias. A model trained on this data wouldn't be "predictive"; it would be discriminatory. It would just learn to copy old, unfair patterns from the data. For example, if women in the past left the company at a higher rate (maybe due to a bad culture), the model would learn Gender=Female $\rightarrow$ High Attrition Risk and penalize new female employees.**
- **Mitigation Strategy: The best strategy here is two-fold:**
    1. **Exclusion: The safest and most ethical approach is to just explicitly exclude 'Gender', 'Marital Status', and any other protected-class features from the dataset before training.**
    2. **Auditing for Proxies: After training the model *without* these features, you must audit it for fairness. You have to check if the model learned the same bias from "proxy" variables (e.g., if 'Job Title' is 90% female, it could become a proxy for 'Gender'). By checking the model's error rates across different groups, you can ensure it's not accidentally discriminating.**

5. Conclusion

This project was a good look at how feature engineering isn't a "one-size-fits-all" process. The techniques for images (transfer learning) and text (NLP pipelines) were completely different. In both cases, the raw data was pretty much unusable. These transformation steps were essential for converting that raw data into a clean, meaningful set of features, which just shows that good feature engineering is the most important foundation for any successful machine learning model.

Github Repo Link:- https://github.com/Arjun3574/-Feature-Engineering-

Google Colab Link :- https://colab.research.google.com/drive/1Tritf-KP4MoSABp6syKxAOphsL_tcaAV?usp=sharing

Name: Arjun Mankuskar PRN:202401110038